Positive Sharing and Abstract Machines

Abstract. Wu's positive λ-calculus is a recent call-by-value λ-calculus with sharing coming from Miller and Wu's study of the proof-theoretical concept of focalization. Accattoli and Wu showed that it simplifies a technical aspect of the study of sharing; namely it rules out the recurrent issue of renaming chains, that often causes a quadratic time slowdown.
In this paper, we define the natural abstract machine for the positive λ-calculus and show that it suffers from an inefficiency: the quadratic slowdown somehow reappears when analyzing the cost of the machine. We then design an optimized machine for the positive λ-calculus, which we prove efficient. The optimization is based on a new slicing technique which is dual to the standard structure of machine environments.

Keywords: λ-calculus · abstract machines · complexity analyses.

1 Introduction

The λ-calculus is a minimalistic abstract setting that does not come with a fixed implementation schema. This is part of its appeal as a theoretical framework. Yet, for the very same reason, many different implementation techniques have been developed along the decades. How to implement the λ-calculus is in fact a surprisingly rich problem with no absolute best answer.

Recently, a proof theoretical study about focusing by Miller and Wu [20] led to a new λ-calculus with sharing—Wu's positive λ-calculus $\lambda_{\mathtt{pos}}$ [26]—that provides a fresh perspective on, and an improvement of a recurrent efficiency issue, as shown by Accattoli and Wu [14]. The present paper studies a somewhat surprising fact related to the efficiency of the positive λ-calculus.

Similarly to the ordinary λ-calculus, Wu's calculus is a minimalistic abstract setting. Its sharing mechanism, however, decomposes β-reduction in micro steps and makes it closer to implementations than the ordinary λ-calculus. It is, in fact, almost an abstract machine.

This work stems from the observation that the natural refinement of $\lambda_{\mathtt{pos}}$ as an abstract machine suffers from an inefficiency. Intuitively, the efficiency issue improved by $\lambda_{\mathtt{pos}}$ resurfaces at the lower level of machines. After pointing out the problem, we design an optimized abstract machine and prove it efficient, solving the issue. The adopted *slicing* optimization is new and based on a dual of the standard environment data structure for abstract machines. We believe it to be an interesting new implementation schema.

The Positive λ-Calculus. In [20], Miller and Wu decorate focalized proofs for minimal intuitionistic logic with proof terms. Minimal intuitionistic logic is the

proof-theoretical counterpart of the λ-calculus via the Curry-Howard correspondence. Focalization is a technique that constrains the shape of proofs depending on a polarity assignment to atomic formulas. Miller and Wu show that uniformly adopting the negative polarity induces the ordinary λ-calculus, while the positive polarity induces an alternative syntax with (sub-term) sharing, where sharing is represented via let-expressions or, equivalently, explicit substitutions.

An important aspect of this new positive syntax is its *compactness property*, also referred to as *positive sharing*: the shape of terms is highly constrained, more than in most other calculi with let-expressions or explicit substitutions. In particular, applications cannot be nested, arguments can only be variables, applications and abstractions are always shared, and variables cannot be shared. Intuitively, the constrained syntax somewhat forces a maximal sharing of subterms by ruling out redundant cases of sharing from the grammar for terms.

In [26], Wu endows the positive syntax with rewriting rules. Because of the compact syntax, there is not much freedom for defining the rules: they have to be call-by-value, and they have to be micro-step, that is, the granularity of the substitution process is the one of abstract machines, that replace one variable occurrence at a time. Call-by-name or meta-level substitution on all occurrences of a variable are indeed ruled out because they do not preserve compactness. The outcome is the positive λ-calculus $\lambda_{\mathtt{pos}}$. The difference between $\lambda_{\mathtt{pos}}$ and an abstract machine is only that $\lambda_{\mathtt{pos}}$ does not have rules searching for redexes.

Renaming Chains. In [14], Accattoli and Wu show two things. Firstly, compactness rules out a recurrent issue in the study of sharing and abstract machines, namely *renaming chains*. In general, when one adds to the λ-calculus an explicit substitution, noted here $t[x \leftarrow u]$ (equivalent to $\mathsf{let}\ x = u\ \mathsf{in}\ t$, and sharing u for x in t), then one can have chains of shared variables of the following form: $t[x_1 \leftarrow x_2] \dots [x_{n-1} \leftarrow x_n]$.

These renaming, or indirection chains, are a recurrent burden of sharing-based systems, as they lead to both time and space inefficiencies—typically a quadratic time slowdown—and need optimizations to avoid their creations, as done for instance by Sands et al. [23], Wand [25], Friedman et al. [17], and Sestoft [24]. In the literature on sharing and abstract machines, the issue tended to receive little attention, until Accattoli and Sacerdoti Coen focused on it [13]. Removing renaming chains is also essential in the study of reasonable logarithmic space for the λ-calculus, see Accattoli et al. [8].

In the positive λ-calculus, variables cannot be shared. Therefore, renaming chains simply cannot be expressed, ruling out the issue. It is worth pointing out, however, that forbidding the sharing of variables requires tuning the rewriting rules by adding some meta-level renamings.

Useful Sharing. Secondly, Accattoli and Wu show that the compactness and the removal of renaming chains of the positive λ-calculus have the effect of drastically simplifying *useful sharing*, a sophisticated implementation technique introduced by Accattoli and Dal Lago in the study of reasonable time cost models [7]. Intuitively, useful sharing aims at preventing the useless unfolding of sharing, which

is exactly what is achieved by compactness via the restriction of the grammar of terms. That is, positive sharing captures the essence of useful sharing.

To be precise, Accattoli and Wu show that compactness simplifies the *specification* of useful sharing, but they do not provide precise cost analyses—that are an essential aspect of useful sharing—for the positive λ-calculus.

The Inefficiency. The inception of the present work is exactly the desire to develop a cost analysis of $\lambda_{\mathtt{pos}}$. Cost analyses are usually done via a study of an abstract machine for the calculus of interest. The somewhat surprising fact is that the natural abstract machine associated to the positive λ-calculus—first given here, and dubbed Natural POsitive Machine, or *Natural POM*—is inefficient. The culprit is the mentioned meta-level renamings added to compensate for the fact that variables cannot be shared. The implementation of these renamings induce a quadratic (rather than linear) overhead in the number of β-steps. Essentially, the inefficiency of renaming chains reappears at the lower level of the Natural POM even if the chains themselves have disappeared.

Useful sharing is meant to reduce the overhead from exponential to polynomial, so the inefficiency does not invalidate the value of $\lambda_{\mathtt{pos}}$ for specifying useful sharing. The literature however contains abstract machines for useful sharing having only a *linear* overhead (by Accattoli and co-authors [12,10,5,6]), thus the Natural POM compares poorly, despite the fact that the positive λ-calculus is a better specification of useful sharing than those in the literature.

The Solution: Sub-Term Property and Slices. We then design an optimized abstract machine, the *Sliced POM*, that solves the issue and recovers linearity in the number of β-steps. To give a hint of the solution, we need to say a bit more about the problem. Complexity analyses of abstract machines are based on their crucial *sub-term property*, stating that all the terms duplicated along the run of the machine are sub-terms of the initial term. The property allows one to bound the cost of each transition using the initial term, which is essential in order to express the cost of the run as a function of the size of the initial term.

Now, the Natural POM does verify the sub-term property *with respect to duplications*; the source of the inefficiency for once is not the duplication process. The source is nonetheless related to a lack of sub-term property *for renamings*: the additional meta-level renamings act over a scope that might not be a sub-term of the initial term. The solution amounts to *slicing* such scopes in slices that are sub-terms of the initial term, and in noting that each meta-level renaming is confined to exactly one slice. The slicing technique is managed very easily, via an additional basic data structure, the slice stack.

Slices vs Environments. A pleasant aspect is that the slicing stack can be seen as *dual* to the environment data structure used to manage sharing:

- an environment entry $[x \leftarrow t]$ stores the delayed substitution of t for x waiting for the evaluation of active term to expose occurrences to replace;
- an entry $t[x \leftarrow \cdot]$ of the slice stack waits for the active term to become a variable z to be substituted for x in the slice t.

This duality suggests that the slicing technique exposes a new natural structure. The meta-level renamings of the calculus are similar to the ones generated by β-redexes. In traditional settings with sharing, indeed, the optimizations to remove renaming chains act on the environment. We find it interesting that positive sharing *disentangles* meta-level renamings from the usual substitution process, and manages them via a sort of dual mechanism.

On the Value of Positive Sharing. The reader might wonder what the value of positive sharing is, given that the issue that it is supposed to solve does reappear at the level of machines and forces the design of a new solution. In our opinion, the value of positive sharing is in re-structuring the study of sharing techniques. The study by Accattoli and Wu [14] suggests that positive sharing *removes* the issue of renaming chains, but our work provides a refined picture. Namely, positive sharing rather enables a neater theory of sharing, disentangling renaming chains from the treatment of explicit substitutions, and encapsulating their inefficiency at the lower level of implementations choices. Additionally, it brings to the fore the concept of slice stack, which we believe is an interesting addition to the theory of implementations of the λ-calculus.

OCaml Implementation. As supplementary material on HotCRP, we provide an implementation in OCaml of the Sliced POM, which is overviewed in Appendix A (p. 20) for lack of space. The implementation is meant to provide further evidence on the cost of the atomic operations of the machine. Moreover, it allows the interested reader to enter ordinary λ-terms and see how they are transformed into positive terms (via the transformation studied by Accattoli and Wu in [14]) and then run by the Sliced POM.

The implementation is a prototype: it does not attempt to further optimize space usage or to recover garbage memory, nor does it optimize the code that is unrelated to running the machine (*e.g.* the pretty-printing of machine states).

Proofs. Omitted proofs are in the Appendix. In case of acceptance, this long version with Appendix will be uploaded on arXiv.

2 The Positive λ-Calculus

In this section, we present Wu's positive λ-calculus [26]. Precisely, we adopt the *explicit open* variant from Accattoli and Wu [14]. For simplicity, we simply refer to it as the positive λ-calculus, and note it λ_{pos}. We slightly depart from the presentation in [14], omitting the garbage collection rule because it can always be postponed, as shown in [14], and changing some notations. The definition is in Fig. 1.

Terms. The positive λ-calculus uses let-expressions, similarly to Moggi's CbV calculus [21,22]. We do however write a let-expression $\mathsf{let}\ x = u\ \mathsf{in}\ t$ as a more compact *explicit substitution* $t[x \leftarrow u]$ (ES for short), which binds x in t. Moreover,

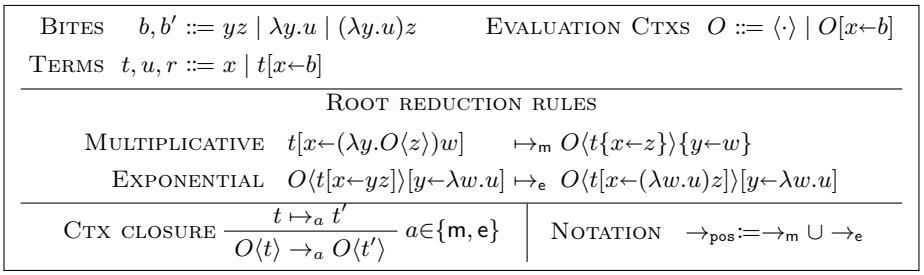

Fig. 1. The positive λ-calculus λ_{pos}.

for the moment our let/ES does not fix an order of evaluation between t and u, in contrast to many papers in the literature (*e.g.* Levy et al. [19]) where u is evaluated first. The evaluation order shall be fixed in the next section.

Additionally, ESs are here used in a slightly unusual way. In λ_{pos}, as in the λ-calculus, there are only three constructors, variables, applications, and abstractions. There are however, various differences, namely:

- Applications have either shape yz or $(\lambda y.t)z$, that is, arguments can only be variables and the left sub-term cannot be an application;
- Applications and abstractions are dubbed *bites* (following the terminology of Accattoli et al. [5]) and are always shared, that is, standalone applications and abstractions are not allowed by the grammar. They can only be introduced by the ESs constructs $[x\leftarrow yz]$, $[x\leftarrow(\lambda y.t)z]$, and $[x\leftarrow\lambda y.u]$;
- Positive sharing is peculiar as positive terms are *not* shared in general, that is, $t[x\leftarrow u]$ is not a positive term. There is no construct for sharing variables or applications/abstractions with top-level sharing, *i.e.* $t[x\leftarrow y]$ or $t[x\leftarrow yz[y\leftarrow\lambda w.u]]$ are not terms of λ_{pos}.

The set of free variables of a term t is denoted by $\mathsf{fv}(t)$ and it is defined as expected. Terms are identified up to α-renaming. We use $t\{x\leftarrow y\}$ for the capture-avoiding substitution of y for each free occurrence of x in t; in this paper we never need the more general operation $t\{x\leftarrow u\}$ substituting terms.

Open Setting. Evaluation in λ_{pos} shall be open in the sense promoted by Accattoli and Guerrieri [9], that is, it does not go under abstraction (also referred to as *weak*) and terms *can* be open (but do not have to). As discussed at length by Accattoli and Guerrieri, this is a more general framework than the standard for functional programming languages of weak evaluation and *closed* terms. The increased generality enables the developed theory to scale up to evaluation under abstraction, which is needed to model proof assistants, by iterating evaluation under abstraction—this is done for instance by Grégoire and Leroy for Coq [18]—because the body of abstractions cannot be assumed to be closed. See [9] for more discussions.

Open Contexts. Open contexts O are simply lists of ESs. As it is immediately seen from the grammar of positive terms, every positive term t can be written uniquely as $O\langle x\rangle$ for some x and O, with O possibly capturing x. If $t = O\langle x\rangle$ then x is referred to as the *head variable* of t.

Rewriting Rules. There are two rewriting rules, following the *at a distance* style promoted by Accattoli and Kesner [11], which involves contexts in the definition of the rules, even before the contextual closure. The rules names come from the connection with linear logic proof nets, which is omitted here.

The multiplicative rule $\to_{\mathsf{m}}$ reduces a shared β-redex. The rule is forced to decompose the body of the abstraction as $O\langle z\rangle$ in order to write the reduct. The point is that the simpler rule $t[x{\leftarrow}(\lambda y.u)w] \mapsto_{\mathsf{m}} t[x{\leftarrow}u[y{\leftarrow}w]]$ does not respect the grammars of the positive λ-calculus, since ESs such as $[x{\leftarrow}u]$ and $[y{\leftarrow}w]$, as well as their nesting, are forbidden.

The exponential rule $\to_{\mathsf{e}}$ simply replaces an applied variable with the associated abstraction. Note that arguments, that are variables, are never replaced, because their replacement would—once more—step out of the grammar of the positive λ-calculus.

Both rules are closed by open contexts and together form the rewriting relation $\to_{\mathsf{pos}}$ of the positive λ-calculus.

Translating λ-Terms. In [14], Accattoli and Wu show how to translate λ-terms to positive terms in a way that induces a simulation of call-by-value evaluation. We refer the interested reader to their work, because in this paper we only deal with positive terms.

Diamond. The defined calculus is non-deterministic. Consider for instance $t := z[x{\leftarrow}yy][z{\leftarrow}(\lambda w.w)y'][y{\leftarrow}\lambda x'.u]$. One has both:

$$\begin{array}{rcll} & t & \to_{\mathsf{m}} & y'[x{\leftarrow}yy][y{\leftarrow}\lambda x'.u] \\ \text{and} & t & \to_{\mathsf{e}} & z[x{\leftarrow}(\lambda x'.u)y][z{\leftarrow}(\lambda w.w)y'][y{\leftarrow}\lambda x'.u] \end{array}$$

The calculus however is confluent, and even more than confluent, it has the diamond property. According to Dal Lago and Martini [15], a relation $\to$ is *diamond* if $u_1 \leftarrow t \to u_2$ imply $u_1 = u_2$ or $u_1 \to r \leftarrow u_2$ for some r. The diamond property expresses a relaxed form of determinism, since it states that different choice cannot change the result *nor the length of evaluation sequences* (note that the diagram closes in either zero or one steps on both sides).

Theorem 1 (Positive diamond, [14]). *Relation $\to_{\mathsf{pos}}$ is diamond.*

Sub-Term Property. When the substitution process is decomposed in micro steps, usually it is possible to bound the cost of each duplication along an evaluation sequence using the size of the initial term of the sequence—this is the sub-term property. It is crucial in order to analyze the cost of evaluation sequences as a function of the size of the initial term and the number of steps, since the main danger of excessive cost usually comes from duplications. The

property does not hold, for instance, for the ordinary λ-calculus (see Accattoli [1, Section 3]) that relies on meta-level (rather than micro-step) substitution. The positive λ-calculus has the sub-term property, that is expressed for values, because they are what is duplicated by the exponential rule.

Lemma 1 (Sub-term property). *Let $t \to_{\mathsf{pos}}^{*} u$ be a reduction sequence. Then $|\lambda x.r| \leq |t|$ for every bite $\lambda x.r$ duplicated by a* e*-step of the sequence.*

Proof. Formally, the proof is a straightforward induction on the length of the reduction sequence, by looking at the last step and using the *i.h.* The following informal observations however are probably enough. Evaluation duplicates variables (in $\to_{\mathsf{m}}$) and abstractions (in $\to_{\mathsf{e}}$). The substitution of variables cannot change the size of abstractions. For abstractions, just note that the replaced variables are out of abstractions, and open contexts never enter abstractions, so that all abstractions of the reduction sequence can be traced back to t (up to α-renaming). □

3 The Right Strategy

Usually, abstract machines are deterministic and implement a deterministic evaluation strategy. Therefore, in this section, we define a deterministic strategy for the positive λ-calculus and prove its basic properties.

We adopt the right(-to-left) strategy $\to_{\mathtt{r}}$ that picks redexes from right to left. It is a standard approach but it turns out that defining it in λ_{pos} is a bit tricky, because of e-steps, where two ESs interact at a distance.

Redex Positions. For calculi at a distance, the notion of position of a redex—which is mandatory to determine the rightmost redex—might not be as expected. For us, a position in a term is simply an open context.

Definition 1 (Redex positions). *When taking into account the contextual closure,* m*-steps and* e*-steps have the following shapes:*

$$\begin{array}{rcccl} t & = & O'\langle r[x{\leftarrow}(\lambda y.O\langle z\rangle)w]\rangle & \to_{\mathsf{m}} & O'\langle O\langle r\{x{\leftarrow}z\}\rangle\{y{\leftarrow}w\}\rangle = u \\ t & = & O'\langle O\langle r[x{\leftarrow}yz]\rangle[y{\leftarrow}\lambda w.q]\rangle & \to_{\mathsf{e}} & O'\langle O\langle r[x{\leftarrow}(\lambda w.q)z]\rangle[y{\leftarrow}\lambda w.q]\rangle = u \end{array}$$

The position of the m*-step is simply given by the closing context O'. The position of the* e*-step is the context $O'\langle O[y{\leftarrow}\lambda w.q]\rangle$.*

The rationale behind the position of e-steps is the idea that they are triggered when one finds the variable to substitute, and not when one finds the abstraction—this is indeed how abstract machines work. This approach is standard in the literature about ESs at a distance, see *e.g.* Accattoli et al. [4].

Example: in $x[x{\leftarrow}yz][x'{\leftarrow}y'z][y'{\leftarrow}\lambda w'.r][y{\leftarrow}\lambda w.u]$ there are two redexes (on y and y') and the rightmost one is on y' (despite the ES on y' occurring to the left of the one on y).

Right Contexts. We specify the strategy via the notion of right contexts, itself specified via the notion of applied free variable. In fact, there are two dual ways of defining right contexts. We present them both and prove their equivalence.

Definition 2 (Applied free variables, right contexts). *The set of applied free variables* $\mathsf{afv}(O)$ *(out of abstractions) of an open context* O *is defined as:*

Set of applied free variables

$$\begin{array}{rcl|rcl} \mathsf{afv}(\langle\cdot\rangle) & \coloneqq & \emptyset & \mathsf{afv}(O[x{\leftarrow}yz]) & \coloneqq & (\mathsf{afv}(O)\setminus\{x\})\cup\{y\} \\ \mathsf{afv}(O[x{\leftarrow}(\lambda y.t)z]) & \coloneqq & \mathsf{afv}(O)\setminus\{x\} & \mathsf{afv}(O[x{\leftarrow}\lambda y.t]) & \coloneqq & \mathsf{afv}(O)\setminus\{x\} \end{array}$$

The two definitions of right contexts (we use on purpose the same meta variable, since they shall be proved equivalent right next) are given by:

Outside-in right contexts

$$R ::= \langle\cdot\rangle \mid R[x{\leftarrow}yz] \mid R[x{\leftarrow}\lambda y.u] \text{ if } x \notin \mathsf{afv}(R)$$

Inside-out right contexts

$$R ::= \langle\cdot\rangle \mid R\langle\langle\cdot\rangle[x{\leftarrow}yz]\rangle \text{ if } R(y) \neq \lambda w.t \mid R\langle\langle\cdot\rangle[x{\leftarrow}\lambda y.u]\rangle$$

Proof p. 23

Lemma 2. *Outside-in and inside-out right contexts coincide.*

Because of the lemma, we only speak of right contexts, and adopt the most convenient definition in each case.

Right Strategy. We now have all the ingredients to define the right strategy and prove its basic properties.

Definition 3 (Right strategy). *A* $\to_{\mathtt{pos}}$ *step is* right *when its position is a right context. The right strategy* $\to_{\mathtt{r}}$ *reduces at each step a right redex, if any. We write* $t \to_{\mathsf{rm}} u$ *(resp.* $t \to_{\mathsf{re}} u$*) for a right* m*-step (resp.* e*-step).*

Proof p. 24

Lemma 3 (Basic properties of the right strategy).

1. Determinism*: if* $t \to_{\mathtt{r}} t_1$ *and* $t \to_{\mathtt{r}} t_2$ *then* $t_1 = t_2$*;*
2. No premature stops*: if* $t \to_{\mathtt{pos}} u$ *then* $t \to_{\mathtt{r}} r$ *for some* r*.*

4 A Natural but Inefficient Positive Machine

In this section, we give the natural abstract machine implementing the right strategy of the previous section, obtained by adding a basic mechanism for searching redexes, discuss the (in)efficiency of the machine, and explain how to modify it as to make it efficient. The tone is slightly informal. The next sections shall formally define and study the modified machine.

Transitions

Active code	Right ctx		Active code	Right ctx	
$t[x{\leftarrow}\lambda y.u]$	$\lhd R$	$\rightsquigarrow_{\mathtt{sea}_1}$	t	$\lhd R\langle\langle\cdot\rangle[x{\leftarrow}\lambda y.u]\rangle$	
$t[x{\leftarrow}yz]$	$\lhd R$	$\rightsquigarrow_{\mathtt{sea}_2}$	t	$\lhd R\langle\langle\cdot\rangle[x{\leftarrow}yz]\rangle$	$(*)$
$t[x{\leftarrow}yz]$	$\lhd R$	$\rightsquigarrow_{\mathtt{e}}$	$t[x{\leftarrow}(\lambda w.u)^{\alpha}z]$	$\lhd R$	$(\#)$
$t[x{\leftarrow}(\lambda y.O\langle z\rangle)w]$	$\lhd R$	$\rightsquigarrow_{\mathtt{m}}$	$O\langle t\{x{\leftarrow}z\}\rangle\{y{\leftarrow}w\}$	$\lhd R$	

$(*)$ if $y \notin \mathsf{dom}(R)$ or $R(y) \neq \lambda w.u$; $(\#)$ If $R(y) = \lambda w.u$.

Fig. 2. The Natural Positive Machine (Natural POM).

The Natural POM. States of the Natural POsitive Machine (Natural POM[1]) are pairs $t \lhd R$ denoting a pointer $\lhd$ inside the structure of the term $u := R\langle t\rangle$ represented by the state. The pointer represents the current position of the machine over u. Initially, the pointer is at the rightmost position, that is, initial states have shape $t \lhd \langle\cdot\rangle$. The Natural POM has the four transitions in Fig. 2. The differences with respect to the positive λ-calculus are that:

- *Search*: there are two search transitions $\mathtt{sea}_1$ and $\mathtt{sea}_2$ to move the pointer, in order to search for redexes, that move $\lhd$ right-to-left (hence the symbol);
- *Names*: there is a more controlled management of α-conversion, which is performed only on the exponential step, when copying abstractions.

Transition $\mathtt{sea}_1$ moves the abstraction from the active code to the right context. Transition $\mathtt{sea}_2$ moves $[x{\leftarrow}yz]$ when y is a free variable (that is, $y \notin \mathsf{dom}(R)$) or when it is bound by an ES in R but not one that contains an abstraction, so that the application yz does not give rise to a multiplicative redex. Note that these two cases are exactly the inside-out definition of right contexts in Def. 2.

Example of Run. As an example of execution of the Natural POM, we consider the first few transitions of the infinite run for the positive representation $x[x{\leftarrow}yy][y{\leftarrow}\lambda z.w[w{\leftarrow}zz]]$ of the paradigmatic looping λ-term $\Omega := (\lambda y.yy)(\lambda z.zz)$:

Active code	Right ctx	
$x[x{\leftarrow}yy][y{\leftarrow}\lambda z.w[w{\leftarrow}zz]]$	$\lhd \langle\cdot\rangle$	$\rightsquigarrow_{\mathtt{sea}_1}$
$x[x{\leftarrow}yy]$	$\lhd [y{\leftarrow}\lambda z.w[w{\leftarrow}zz]]$	$\rightsquigarrow_{\mathtt{e}}$
$x[x{\leftarrow}(\lambda z'.w'[w'{\leftarrow}z'z'])y]$	$\lhd [y{\leftarrow}\lambda z.w[x{\leftarrow}zz]]$	$\rightsquigarrow_{\mathtt{m}}$
$w'[w'{\leftarrow}yy]$	$\lhd [y{\leftarrow}\lambda z.w[x{\leftarrow}zz]]$	$\rightsquigarrow_{\mathtt{e}}$ $\quad\ldots$

Basics of the Complexity of Abstract Machines. The problem with the Natural POM concerns the complexity of its overhead with respect to the calculus. Let us first recall the basics of the topic. For abstract machines, the complexity of the overhead for a machine run r of initial term t is measured with respect to

[1] Acronyms for abstract machines tend to end with AM for Abstract Machine. We avoided PAM, however, because there already is a PAM (Pointer Abstract Machine) in the literature, introduced by Danos et al. [16].

two parameters: the number of steps of the underlying calculus and the size $|t|$ of the initial term. A large number of machines has complexity linear in both parameters—shortened to *bi-linear*—as first shown by Accattoli et al. [2], and repeatedly verified after that for even more machines [12,3,5,10,6]. The bi-linearity of the overhead then becomes a design principle, or, when it fails, a strong indication of an inefficiency, and that the machine can be improved.

The key property enabling complexity analyses of machines is the sub-term property already discussed for λ_{pos}. It ensures that the size of states cannot grow more than the size of the initial term at each transition. In turn, this fact usually implies that the time cost is also linearly bounded.

The Inefficiency of the Natural POM. The natural POM inherits the sub-term property from the calculus. Its inefficiency is related to the multiplicative transition $\rightsquigarrow_{\mathsf{m}}$, namely to the meta-level renaming $t\{x{\leftarrow}z\}$ (referring to Fig. 2). Meta-level substitutions are possibly costly. Usually, the danger is the time spent in making copies of the term to duplicate, which might be big and/or because many copies of it might be required. The size of the term to duplicate is not the problem here, since the involved term is a simple variable z.

The culprit actually is the *size of the scope* over which the renaming can take place, rather than the number of copies. There are two renamings. The renaming $\{y{\leftarrow}w\}$ is harmless: its scope seems to be $O\langle t\{x{\leftarrow}z\}\rangle$ but in fact y can occur only in $O\langle z\rangle$ (which is the body of the abstraction of y in the source state of the transition), and the sub-term property guarantees that the size of $O\langle z\rangle$ is bound by the size of the initial term. Therefore, one can propagate $\{y{\leftarrow}w\}$ on $O\langle z\rangle$ before computing the full reduct, staying within a linear cost.

For the renaming $t\{x{\leftarrow}z\}$, however, there is in general no connection between t and the initial term. For the first multiplicative step of the run, t is a sub-term of the initial term. But the step itself re-combines O and t as to create a new term unrelated to the initial one. Consider for instance the following run, where we assume that O captures z and that $(\lambda y.(O\langle z\rangle[z'{\leftarrow}b]))^{\alpha} = \lambda y'.(O'\langle z'\rangle[z''{\leftarrow}b'])$:

$$
\begin{array}{lll}
 & \textsc{Active code} & \textsc{Right ctx} \\
\hline
 & t[x{\leftarrow}x'w][x'{\leftarrow}\lambda y.(O\langle z\rangle[z'{\leftarrow}b])] & \lhd\ \langle\cdot\rangle \\
\rightsquigarrow_{\mathsf{sea}_1} & t[x{\leftarrow}x'w] & \lhd\ [x'{\leftarrow}\lambda y.(O\langle z\rangle[z'{\leftarrow}b])] \\
\rightsquigarrow_{\mathsf{e}} & t[x{\leftarrow}(\lambda y'.(O'\langle z'\rangle[z''{\leftarrow}b']))w] & \lhd\ [x'{\leftarrow}\lambda y.(O\langle z\rangle[z'{\leftarrow}b])] \\
\rightsquigarrow_{\mathsf{m}} & O'\langle t\{x{\leftarrow}z'\}\rangle[z''{\leftarrow}b']\{y'{\leftarrow}w\} & \lhd\ [x'{\leftarrow}\lambda y.(O\langle z\rangle[z'{\leftarrow}b])]
\end{array}
\tag{1}
$$

Now, if the bite b' is a β-redex then the next transition is multiplicative and it is going to rename over $O'\langle t\{x{\leftarrow}z'\}\rangle$ which is not a sub-term of the initial term.

How Big is the Inefficiency? When the time cost depends on something that is not a sub-term of the initial term, things can easily escalate up to exponential costs, as in the paradigmatic case of size explosion, see Accattoli [1, Section 3]. Luckily, here things do not go sideways, there is only a mild inefficiency. The key point is that the sub-term property for duplications *does hold*: it ensures that the size of the whole state grows bi-linearly with the number of transitions,

so there is no exponential growth. The problematic renaming—and thus each multiplicative transition—might then have to scan a scope that is at worst bilinear in the length of the preceding run and the size of the initial term. A standard argument then gives a quadratic bound (in the number of steps and the size of the initial term) on the global cost of all multiplicative steps.

Let us give an example. We use a diverging term because it is the simplest example showcasing the phenomenon. We define the positive representation of $\Omega_3 \coloneqq \delta_3\delta_3$ where $\delta_3 \coloneqq \lambda x.x(xx)$. Let $\tau_3 \coloneqq x[x{\leftarrow}yz][z{\leftarrow}yy]$ and let $\tau_3' \coloneqq x'[x'{\leftarrow}y'z'][z'{\leftarrow}y'y']$ and so on. The analogous of Ω_3 is $\tau_3[y{\leftarrow}\lambda y.\tau_3]$, that runs as follows:

Active code	Right ctx	
$x[x{\leftarrow}yz][z{\leftarrow}yy][y{\leftarrow}\lambda y.\tau_3]$	$\lhd\ \langle\cdot\rangle$	$\rightsquigarrow_{\mathtt{sea}_1}$
$x[x{\leftarrow}yz][z{\leftarrow}yy]$	$\lhd\ [y{\leftarrow}\lambda y.\tau_3]$	$\rightsquigarrow_{\mathtt{e}}$
$x[x{\leftarrow}yz][z{\leftarrow}(\lambda y'.\tau_3')y]$	$\lhd\ [y{\leftarrow}\lambda y.\tau_3]$	$=$
$x[x{\leftarrow}yz][z{\leftarrow}(\lambda y'.x'[x'{\leftarrow}y'z'][z'{\leftarrow}y'y'])y]$	$\lhd\ [y{\leftarrow}\lambda y.\tau_3]$	$\rightsquigarrow_{\mathtt{m}}$
$x[x{\leftarrow}yx'][x'{\leftarrow}yz'][z'{\leftarrow}yy]$	$\lhd\ [y{\leftarrow}\lambda y.\tau_3]$	$\rightsquigarrow_{\mathtt{e}}$
$x[x{\leftarrow}yx'][x'{\leftarrow}yz'][z'{\leftarrow}(\lambda y''.\tau_3'')y]$	$\lhd\ [y{\leftarrow}\lambda y.\tau_3]$	$\rightsquigarrow_{\mathtt{m}}$
$x[x{\leftarrow}yx'][x'{\leftarrow}yx''][x''{\leftarrow}yz''][z''{\leftarrow}yy]$	$\lhd\ [y{\leftarrow}\lambda y.\tau_3]$	$\rightsquigarrow_{\mathtt{e}}$ $\quad\ldots$

It is clear that the scopes of the renamings keep growing, even if the number of renamed occurrences by each renaming is constant.

Removing the Inefficiency: Slices. An observation about the run (1) above suggests how to improve the situation. The idea is to delay the merging of t and $O'\langle z'\rangle$ as $O'\langle t\{x{\leftarrow}z'\}\rangle$ (similarly to how ESs delay meta-level substitutions) by putting the pair (t, x), dubbed *slice* and that shall actually be denoted with $t[x{\leftarrow}\cdot]$, in a new stack storing delayed merges, and keeping as active code $O'\langle z'\rangle$. The observation is that if b' is a β-redex and generates a renaming of z'' then one only needs to inspect $O'\langle z'\rangle$—which, crucially, is a sub-term of the initial term—because z'' cannot occur in t, given that it comes from the body of an abstraction out of t.

Before defining the Sliced POM, and prove that slices do solve the problem, we start over with a more formal approach to abstract machines.

5 Preliminaries About Abstract Machines

In this section, we fix the terminology about abstract machines. We follow Accattoli and co-authors [12,3,5,10,6], adapting their notions to our framework. We mostly stay abstract. In the next section, we shall instantiate the abstract notions on a specific machine.

Abstract Machines Glossary. Abstract machines manipulate *pre-terms*, that is, terms without implicit α-renaming. In this paper, an *abstract machine* is a quadruple $\mathsf{M} = (\mathtt{States}, \rightsquigarrow, \cdot \lhd \cdot, \bar{\cdot})$ the components of which are as follows.

- *States.* A state $\mathtt{Q} \in \mathtt{States}$ is composed by the *active term* t, plus some data structures; the machine of the next section shall have two data structures. Terms in states are actually pre-terms.
- *Transitions.* The pair $(\mathtt{States}, \rightsquigarrow)$ is a transition system with transitions $\rightsquigarrow$ partitioned into *principal transitions*, whose union is noted $\rightsquigarrow_{\mathtt{pr}}$ and that are meant to correspond to rewriting steps on the calculus, and *search transitions*, whose union is noted $\rightsquigarrow_{\mathtt{sea}}$, that take care of searching for (principal) redexes.
- *Initialization.* The component $\triangleleft \subseteq \Lambda \times \mathtt{States}$ is the *initialization relation* associating terms to initial states. It is a *relation* and not a function because $t \triangleleft \mathtt{Q}$ maps a term t (considered modulo α) to a state $\mathtt{Q}$ having a *pre-term representant* of t (which is not modulo α) as active term. Intuitively, any two states $\mathtt{Q}$ and $\mathtt{Q}'$ such that $t \triangleleft \mathtt{Q}$ and $t \triangleleft \mathtt{Q}'$ are α-equivalent. A state $\mathtt{Q}$ is *reachable* if it can be reached starting from an initial state, that is, if $\mathtt{Q}' \rightsquigarrow^* \mathtt{Q}$ where $t \triangleleft \mathtt{Q}'$ for some t and $\mathtt{Q}'$, shortened as $t \triangleleft \mathtt{Q}' \rightsquigarrow^* \mathtt{Q}$.
- *Read-back.* The read-back function $\overline{\cdot} : \mathtt{States} \to \Lambda$ turns reachable states into terms and satisfies the *initialization constraint*: if $t \triangleleft \mathtt{Q}$ then $\overline{\mathtt{Q}} =_\alpha t$.

Further Terminology and Notations. A state is *final* if no transitions apply. A *run* $r : \mathtt{Q} \rightsquigarrow^* \mathtt{Q}'$ is a possibly empty finite sequence of transitions, the length of which is noted $|r|$; note that the first and the last states of a run are not necessarily initial and final. If a and b are transitions labels (that is, $\rightsquigarrow_a \subseteq \rightsquigarrow$ and $\rightsquigarrow_b \subseteq \rightsquigarrow$) then $\rightsquigarrow_{a,b} := \rightsquigarrow_a \cup \rightsquigarrow_b$ and $|r|_a$ is the number of a transitions in r.

Well-Boundness and Renamings. For the machine in this paper, the pre-terms in initial states shall be *well-bound*, that is, they have pairwise distinct bound names; for instance $w[w{\leftarrow}\lambda z.z][x{\leftarrow}\lambda y.y]$ is well-bound while $w[w{\leftarrow}\lambda y.y][x{\leftarrow}\lambda y.y]$ is not. We shall also write t^α in a state $\mathtt{Q}$ for a *fresh well-bound renaming* of t, *i.e.* t^α is α-equivalent to t, well-bound, and its bound variables are fresh with respect to those in t and in the other components of $\mathtt{Q}$.

Mechanical Bismulations. Machines are usually showed to be correct with respect to a strategy via some form of bisimulation relating terms and states. The notion that we adopt is here dubbed *mechanical bisimulation*. The definition, tuned towards complexity analyses, requires a perfect match between the steps of the evaluation sequence and the principal transitions of the machine run.

Definition 4 (Mechanical bisimulation). *A machine* $\mathsf{M} = (\mathtt{States}, \rightsquigarrow, \cdot\triangleleft\cdot, \overline{\cdot})$ *and a strategy* $\to_{\mathtt{str}}$ *on terms are mechanical bisimilar when, given an initial state* $t \triangleleft \mathtt{Q}$*:*

1. Runs to evaluations*: for any run* $r : t \triangleleft \mathtt{Q} \rightsquigarrow^* \mathtt{Q}'$ *there exists an evaluation* $e : t \to^*_{\mathtt{str}} \overline{\mathtt{Q}'}$*;*
2. Evaluations to runs*: for every evaluation* $e : t \to^*_{\mathtt{str}} u$ *there exists a run* $r : t \triangleleft \mathtt{Q} \rightsquigarrow^* \mathtt{Q}'$ *such that* $\overline{\mathtt{Q}'} = u$*;*

Slice stacks	Environments	States	Initialization
$\mathtt{S} ::= \epsilon \mid \mathtt{S} : t[x\leftarrow\cdot]$	$\mathtt{E} ::= \epsilon \mid [x\leftarrow b] : \mathtt{E}$	$\mathtt{Q} ::= (\mathtt{S}, t, \mathtt{E})$	$t \lhd (\epsilon, t^{\alpha}, \epsilon) \quad (*)$

Transitions

Sl. st.	Active slice	Env		Sl. st.	Active slice	Env	
$\mathtt{S}$	$t[x\leftarrow\lambda y.u]$	$\mathtt{E}$	$\rightsquigarrow_{\mathtt{sea}_1}$	$\mathtt{S}$	t	$[x\leftarrow\lambda y.u] : \mathtt{E}$	
$\mathtt{S}$	$t[x\leftarrow yz]$	$\mathtt{E}$	$\rightsquigarrow_{\mathtt{sea}_2}$	$\mathtt{S}$	t	$[x\leftarrow yz] : \mathtt{E}$	(%)
$\mathtt{S}$	$t[x\leftarrow yz]$	$\mathtt{E}$	$\rightsquigarrow_{\mathtt{e}}$	$\mathtt{S}$	$t[x\leftarrow(\lambda w.u)^{\alpha} z]$	$\mathtt{E}$	(#)(*)
$\mathtt{S}$	$t[x\leftarrow(\lambda y.u)w]$	$\mathtt{E}$	$\rightsquigarrow_{\mathtt{m}}$	$\mathtt{S} : t[x\leftarrow\cdot]$	$u\{y\leftarrow w\}$	$\mathtt{E}$	
$\mathtt{S} : t[x\leftarrow\cdot]$	z	$\mathtt{E}$	$\rightsquigarrow_{\mathtt{sea}_3}$	$\mathtt{S}$	$t\{x\leftarrow z\}$	$\mathtt{E}$	

(*) t^{α} is any well-bound code α-equivalent to t such that its bound names are fresh with respect to those in the rest of the state;

(%) if $y \notin \mathsf{dom}(\mathtt{E})$ or $\mathtt{E}(y) \neq \lambda w.u$; (#) If $\mathtt{E}(x) = \lambda w.u$.

Read back

Envs (to ctxs)	$\overline{\epsilon} := \langle\cdot\rangle$	$\overline{[x\leftarrow b] : \mathtt{E}} := \overline{\mathtt{E}}\langle\langle\cdot\rangle[x\leftarrow b]\rangle$
States (to terms)	$\overline{(\epsilon, t, \mathtt{E})} := \overline{\mathtt{E}}\langle t\rangle$	$\overline{(\mathtt{S} : t[x\leftarrow\cdot], O\langle y\rangle, \mathtt{E})} := \overline{(\mathtt{S}, O\langle t\{x\leftarrow y\}\rangle, \mathtt{E})}$

Fig. 3. The Sliced Positive Machine (Sliced POM).

3. Principal matching*: for every principal transition $\rightsquigarrow_{\mathtt{a}}$ of label a of* M*, in both previous points the number $|r|_{\mathtt{a}}$ of* $\mathtt{a}$*-transitions in r is exactly the number $|e|_a$ of of* $\mathtt{a}$*-steps in the evaluation e, i.e. $|e|_{\mathtt{a}} = |r|_{\mathtt{a}}$.*

The proof that a machine and a strategy are in a mechanical bisimulation follows from some basic properties, grouped under the notion of *distillery*, following Accattoli et al. [2] (but removing their use of structural equivalence, that here is not needed).

Definition 5 (Distillery). *A machine* $\mathsf{M} = (\mathtt{States}, \rightsquigarrow, \cdot \lhd \cdot, \bar{\cdot})$ *and a strategy* $\rightarrow_{\mathtt{str}}$ *are a* distillery *if the following conditions hold:*

1. Principal projection*: $\mathtt{Q} \rightsquigarrow_{\mathtt{a}} \mathtt{Q}'$ implies $\overline{\mathtt{Q}} \rightarrow_{\mathtt{a}} \overline{\mathtt{Q}'}$ for every principal transition of label* $\mathtt{a}$*;*
2. Search transparency*: $\mathtt{Q} \rightsquigarrow_{\mathtt{sea}} \mathtt{Q}'$ implies $\overline{\mathtt{Q}} = \overline{\mathtt{Q}'}$;*
3. Search transitions terminate*: $\rightsquigarrow_{\mathtt{sea}}$ terminates;*
4. Determinism*: $\rightarrow_{\mathtt{str}}$ is deterministic;*
5. Halt*: M final states decode to $\rightarrow_{\mathtt{str}}$-normal terms.*

Theorem 2 (Sufficient condition for mechanical bisimulations). *Let a machine* M *and a strategy* $\rightarrow_{\mathtt{str}}$ *be a distillery. Then, they are mechanical bisimilar.* Proof p. 25

6 The Sliced Positive Machine

In this section, we present the optimized machine for the positive λ-calculus.

The Sliced POM: Data Structures. The Sliced Positive Machined is defined in Fig. 3. The machine has two data structures, the environment $\mathtt{E}$ and the slice stack $\mathtt{S}$. The active code is re-dubbed *active slice*.

Environments $\mathtt{E}$ are nothing else but encodings of right contexts R. We prefer to change the representation for two reasons. Firstly, it is closer to how contexts are represented in the OCaml implementation. Secondly, it is a bit more precise: environments are free structures, and an invariant shall prove that they decode to right contexts (which are defined via some additional conditions).

The slice stack is simply a list of slices $t[x\leftarrow\cdot]$. Note the duality between environments and slice stacks: the entries of both are pairs of a term t and a variable x, but environment entries are meant to substitute t for x on some other term, while slices are meant to receive a variable and substitute it for x in t.

The Sliced POM: New Transition. The Sliced POM inherits the same four transitions of the Natural POM, but note that in transition $\rightsquigarrow_{\mathsf{m}}$ now it is no longer necessary to decompose the body u of the abstraction as $O\langle z\rangle$ (please note that u is a meta variable for *terms*, and not for variables). Additionally, the Sliced POM has a new search transition $\mathtt{sea}_3$. When the evaluation of the active slice is over, which happens when one is left only with the head variable z of the slice, the new transitions $\mathtt{sea}_3$ pops the first slice on the slice stack and replaces z for x in t. An invariant shall guarantee that t is a sub-term of the initial term, so that the cost of the renaming now is under control.

A Technical Point. The attentive reader might wonder why $\rightsquigarrow_{\mathsf{m}}$ and $\rightsquigarrow_{\mathtt{sea}_3}$ are not reformulated in the following eager way (with slices reduced to be simply terms), where the head of the abstraction is substituted when the slice is *pushed* on the slice stack, and not when it is popped:

Sl. st.	Active slice	Env		Sl. st.	Active slice	Env
$\mathtt{S}$	$t[x\leftarrow(\lambda y.O\langle z\rangle)w]$	$\mathtt{E}$	$\rightsquigarrow_{\mathsf{m}}$	$\mathtt{S} : t\{x\leftarrow z\}$	$O\langle z\rangle\{y\leftarrow w\}$	$\mathtt{E}$
$\mathtt{S} : t$	z	$\mathtt{E}$	$\rightsquigarrow_{\mathtt{sea}_3}$	$\mathtt{S}$	t	$\mathtt{E}$

We do not adopt the eager approach because that the evaluation of the active slice might change its head variable (thus making it unsound to substitute eagerly), as it is demonstrated by the following run of the standard Sliced POM:

Slice stack	Active slice	Env	
ϵ	$t[x\leftarrow(\lambda y.z[z\leftarrow(\lambda x'.x')w'])w]$	ϵ	$\rightsquigarrow_{\mathsf{m}}$
$\epsilon : t[x\leftarrow\cdot]$	$z[z\leftarrow(\lambda x'.x')w']$	ϵ	$\rightsquigarrow_{\mathsf{m}}$
$\epsilon : t[x\leftarrow\cdot] : z[z\leftarrow\cdot]$	w'	ϵ	$\rightsquigarrow_{\mathtt{sea}_3}$
$\epsilon : t[x\leftarrow\cdot]$	w'	ϵ	

7 Mechanical Bisimulation

In this section, we establish the mechanical bisimulation between the right strategy $\rightarrow_{\mathtt{r}}$ of the positive λ-calculus and the Sliced POM, by proving that the two form a distillery.

Invariants. The proof of the properties defining a distillery are based on the following invariants of the machine.

Lemma 4 (Qualitative invariants). *Let* $\mathtt{Q} = (\mathtt{S}, t, \mathtt{E})$ *be a reachable state.*

1. Contextual read-back*:* $\overline{\mathtt{E}}$ *is a right context.*
2. Well-bound*:*
   - Bound names in terms*: if* $\lambda x.u$ *or* $u[x{\leftarrow}b]$ *occurs in* $\mathtt{Q}$ *and* x *has any other occurrence in* $\mathtt{Q}$ *then it is as a free variable of* u*;*
   - ES names of environment entries*: for any ES* $[y{\leftarrow}b]$ *in* $\mathtt{E}$ *the name* y *can occur (in any form) only on the left of that ES in* $\mathtt{Q}$*.*

Proof. By induction on the length of the run reaching $\mathtt{Q}$. For both points, the base case trivially holds, and the inductive case is by analysis of the last transition, which is always a straightforward inspection of the transitions using the *i.h.* For contextual read-back, the proof relies on the inside-out definition of right contexts (see Def. 2). □

The well-bound invariant has two consequences that might not be evident in the way it is stated. Firstly, in a state there cannot be two ESs $[x{\leftarrow}b]$ and $[x{\leftarrow}b']$ on the same variable. This fact implies the determinism of the machine, that however shall not be proved because it is not necessary (determinism of the strategy is enough for proving the bisimulation). Secondly, the name x of an environment entry $[x{\leftarrow}b]$ can occur both in the active slice and in the slice stack, while the names of ESs in slices can occur only within the slice. In particular, x cannot occur in $\mathtt{S}$ in $(\mathtt{S}, t[x{\leftarrow}b], \mathtt{E})$.

Theorem 3 (Distillery). Proof p. 26

1. *Principal projection:*
   (a) *if* $\mathtt{Q} \rightsquigarrow_{\mathtt{e}} \mathtt{Q}'$ *then* $\overline{\mathtt{Q}} \to_{\mathtt{e}} \overline{\mathtt{Q}'}$.
   (b) *if* $\mathtt{Q} \rightsquigarrow_{\mathtt{m}} \mathtt{Q}'$ *then* $\overline{\mathtt{Q}} \to_{\mathtt{m}} \overline{\mathtt{Q}'}$.
2. *Search transparency: if* $\mathtt{Q} \rightsquigarrow_{\mathtt{sea}_1,\mathtt{sea}_2,\mathtt{sea}_3} \mathtt{Q}'$ *then* $\overline{\mathtt{Q}} = \overline{\mathtt{Q}'}$.
3. *Search terminates:* $\rightsquigarrow_{\mathtt{sea}_1,\mathtt{sea}_2,\mathtt{sea}_3}$ *is terminating.*
4. *Halt: if* $\mathtt{Q}$ *is final then it is of the form* $(\epsilon, x, \mathtt{E})$ *and* $\overline{\mathtt{Q}} = \overline{\mathtt{E}}\langle x\rangle$ *is* $\to_{\mathtt{pos}}$*-normal.*

The points of the proved theorem together with determinism of the right strategy (Lemma 3) provide all the requirements for a distillery. Then, the abstract theorem about distilleries (Thm. 2) gives the following corollary.

Corollary 1 (Mechanical bisimulation). *The Sliced POM and the right strategy* $\to_{\mathtt{r}}$ *are in a mechanical bisimulation.*

8 Complexity Analysis

In this section, we show that the Sliced POM can be concretely implemented within a bi-linear overhead, that is, linear in the number of m-steps/transitions and the size of the initial term.

Sub-Term Property. As it is standard, the complexity analysis crucially relies on the sub-term property. In CbV settings, the property is usually expressed saying that all values are sub-terms of the initial terms, as in Lemma 1. The Sliced POM only duplicates values too, but (as we have discussed for the Natural POM) we also want to know that the terms to which renamings are applied are sub-terms of the initial term, and these terms are not values. Thus, the property is given with respect to *all* terms in a state.

There is in fact a very minor exception. The substitution performed by an exponential transition takes two sub-terms of the initial term, namely $t[x \leftarrow yz]$ and $\lambda w.u$, and creates a term $t[x \leftarrow (\lambda w.u)^{\alpha} z]$ that is not a sub-term of the initial one. The new term, however, is very short lived: the next transition is multiplicative and decomposes that term in sub-terms of the initial term (up to renaming).

Lemma 5 (Sub-term property). *Let $t \triangleleft \mathtt{Q} \rightsquigarrow^* \mathtt{Q}' = (\mathtt{S}, u, \mathtt{E})$ be a Sliced POM run. Then:*

1. *If $\mathtt{Q}'$ is not the target of a e-transition then $|r| \leq |t|$ for any term r in $\mathtt{Q}'$;*
2. *Otherwise, $|r| \leq |t|$ for any term r in $\mathtt{Q}'$ except u.*

Proof. By induction on the length of the run, inspecting the last transition and using the *i.h.* □

Number of Transitions. Some basic observations about the transitions, together with the sub-term property, allow us to bound their number using the two key parameters (that is, number of m-steps/transitions and size of the initial term).

Lemma 6 (Number of transitions). *Let $r : t \triangleleft \mathtt{Q} \rightsquigarrow^* \mathtt{Q}'$ be a Sliced POM run.*

1. *$|r|_{\mathsf{e},\mathtt{sea}_3} \in \mathcal{O}(|r|_{\mathsf{m}})$;*
2. *$|r|_{\mathtt{sea}_1,\mathtt{sea}_2} \in \mathcal{O}(|t| \cdot (|r|_{\mathsf{m}} + 1))$.*

Proof.

1. $|r|_{\mathsf{e}} \leq |r|_{\mathsf{m}} + 1$ because exponential transitions can only be followed by multiplicative transitions.
   $|r|_{\mathtt{sea}_3} \leq |r|_{\mathsf{m}}$ because every $\mathtt{sea}_3$ transition consumes one entry from the slice stack, which are created only by m transitions.
2. Note that $\mathtt{sea}_1/\mathtt{sea}_2$ transitions decrease the size of the active slice, which is increased only by transitions e and $\mathtt{sea}_3$. By the sub-term property (Lemma 5), the active slice increase by transitions e and $\mathtt{sea}_3$ is bounded by the size $|t|$ of the initial term. By Point 1, $|r|_{\mathtt{sea}_3,\mathsf{e}} = \mathcal{O}(|r|_{\mathsf{m}})$. Then $|r|_{\mathtt{sea}_1,\mathtt{sea}_2} \in \mathcal{O}(|t| \cdot (|r|_{\mathsf{e},\mathtt{sea}_2} + 1)) = \mathcal{O}(|t| \cdot (|r|_{\mathsf{m}} + 1))$. □

Cost of Single Transitions. Lastly, we need some assumptions on how the Sliced POM can be concretely implemented. Transition $\mathtt{sea}_2$ can evidently be done in $\mathcal{O}(1)$. Our OCaml implementation—overviewed in Appendix A (p. 20)—represents variables as memory locations and variable occurrences as pointers to those locations, obtaining random access to environment entries in $\mathcal{O}(1)$.

Therefore, also transition $\mathtt{sea}_1$ can be done in $\mathcal{O}(1)$. The cost of transition $\mathtt{e}$ is bound by the size of the value to copy, itself bound by the sub-term property. The cost of transitions $\mathtt{m}$ and $\mathtt{sea}_3$ is bound by the size of the term to rename, itself bound by the sub-term property. The next lemma sums it up.

Lemma 7 (Cost of single transitions). *Let $t \triangleleft \mathtt{Q} \rightsquigarrow^* \mathtt{Q}'$ be a Sliced POM run. Implementing $\mathtt{sea}_1$ and $\mathtt{sea}_2$ transitions from $\mathtt{Q}'$ costs $\mathcal{O}(1)$ each while implementing $\mathtt{e}$, $\mathtt{m}$, and $\mathtt{sea}_3$ transitions costs $\mathcal{O}(|t|)$ each.*

Putting all together, we obtain our main result: a bilinear bound for the Sliced POM, showing that it is an efficient machine for the right strategy.

Theorem 4 (The Sliced POM is bi-linear). *Let $r : t \triangleleft \mathtt{Q} \rightsquigarrow^* \mathtt{Q}'$ be a Sliced POM run. Then r can be implemented on random access machines in $\mathcal{O}(|t| \cdot (|r|_{\mathtt{m}} + 1))$.*

Proof. The cost of implementing r is obtained by multiplying the number of each kind of transitions (Lemma 6) by the cost of that kind of transition (Lemma 7), and summing over all kinds of transition. □

9 Conclusions

In λ-calculi with sharing, renaming chains are a recurrent issue that causes both time and space inefficiencies. The recently introduced positive λ-calculus removes renaming chains, while adding some meta-level renamings. This paper stems from the observation that the meta-level renamings reintroduce a time inefficiency if implemented naively.

The problem is analyzed via the sub-term property, showing that the culprit is the fact that the scope of renamings is not a sub-term of the initial term. The analysis leads to the design of an optimized machine, the new Sliced POM, that removes once and for all the inefficiency. The key tool is a new decomposition in slices of the scopes of renamings, via a new stack for slices playing a role dual to that of environments. We also provide a prototype OCaml implementation of the Sliced POM, described in Appendix A (p. 20).

Future Work. At the theoretical level, we plan to adapt the positive λ-calculus and the Sliced POM to call-by-need evaluation. At the practical level, it would be interesting to see how the schema of the Sliced POM combines with other implementation techniques such as closure conversion or skeletal call-by-need. We also suspect that the Sliced POM can be used to simplify the sophisticated machine for Strong Call-by-Value by Accattoli et al. in [6].

References

1. Accattoli, B.: Exponentials as substitutions and the cost of cut elimination in linear logic. Log. Methods Comput. Sci. **19**(4) (2023). https://doi.org/10.46298/LMCS-19(4:23)2023
2. Accattoli, B., Barenbaum, P., Mazza, D.: Distilling abstract machines. In: 19th ACM SIGPLAN International Conference on Functional Programming, ICFP 2014. pp. 363–376. ACM (2014). https://doi.org/10.1145/2628136.2628154
3. Accattoli, B., Barenbaum, P., Mazza, D.: A strong distillery. In: Feng, X., Park, S. (eds.) Programming Languages and Systems - 13th Asian Symposium, APLAS 2015, Pohang, South Korea, November 30 - December 2, 2015, Proceedings. Lecture Notes in Computer Science, vol. 9458, pp. 231–250. Springer (2015). https://doi.org/10.1007/978-3-319-26529-2_13
4. Accattoli, B., Bonelli, E., Kesner, D., Lombardi, C.: A nonstandard standardization theorem. In: Jagannathan, S., Sewell, P. (eds.) The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '14, San Diego, CA, USA, January 20-21, 2014. pp. 659–670. ACM (2014). https://doi.org/10.1145/2535838.2535886
5. Accattoli, B., Condoluci, A., Guerrieri, G., Sacerdoti Coen, C.: Crumbling abstract machines. In: Komendantskaya, E. (ed.) Proceedings of the 21st International Symposium on Principles and Practice of Programming Languages, PPDP 2019, Porto, Portugal, October 7-9, 2019. pp. 4:1–4:15. ACM (2019). https://doi.org/10.1145/3354166.3354169
6. Accattoli, B., Condoluci, A., Sacerdoti Coen, C.: Strong call-by-value is reasonable, implosively. In: 36th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2021, Rome, Italy, June 29 - July 2, 2021. pp. 1–14. IEEE (2021). https://doi.org/10.1109/LICS52264.2021.9470630
7. Accattoli, B., Dal Lago, U.: (leftmost-outermost) beta reduction is invariant, indeed. Log. Methods Comput. Sci. **12**(1) (2016). https://doi.org/10.2168/LMCS-12(1:4)2016
8. Accattoli, B., Dal Lago, U., Vanoni, G.: Reasonable space for the λ-calculus, logarithmically. In: Baier, C., Fisman, D. (eds.) LICS '22: 37th Annual ACM/IEEE Symposium on Logic in Computer Science, Haifa, Israel, August 2 - 5, 2022. pp. 47:1–47:13. ACM (2022). https://doi.org/10.1145/3531130.3533362
9. Accattoli, B., Guerrieri, G.: Open call-by-value. In: Igarashi, A. (ed.) Programming Languages and Systems - 14th Asian Symposium, APLAS 2016, Hanoi, Vietnam, November 21-23, 2016, Proceedings. Lecture Notes in Computer Science, vol. 10017, pp. 206–226 (2016). https://doi.org/10.1007/978-3-319-47958-3_12
10. Accattoli, B., Guerrieri, G.: Abstract machines for open call-by-value. Sci. Comput. Program. **184** (2019). https://doi.org/10.1016/J.SCICO.2019.03.002
11. Accattoli, B., Kesner, D.: The structural *lambda*-calculus. In: Dawar, A., Veith, H. (eds.) Computer Science Logic, 24th International Workshop, CSL 2010, 19th Annual Conference of the EACSL, Brno, Czech Republic, August 23-27, 2010. Proceedings. Lecture Notes in Computer Science, vol. 6247, pp. 381–395. Springer (2010). https://doi.org/10.1007/978-3-642-15205-4_30
12. Accattoli, B., Sacerdoti Coen, C.: On the relative usefulness of fireballs. In: 30th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2015, Kyoto, Japan, July 6-10, 2015. pp. 141–155. IEEE Computer Society (2015). https://doi.org/10.1109/LICS.2015.23

13. Accattoli, B., Sacerdoti Coen, C.: On the value of variables. Information and Computation **255**, 224–242 (2017). https://doi.org/10.1016/j.ic.2017.01.003
14. Accattoli, B., Wu, J.H.: Positive focusing is directly useful. Electronic Notes in Theoretical Informatics and Computer Science **Volume 4 - Proceedings of MFPS XL**, 3 (Dec 2024). https://doi.org/10.46298/entics.14758
15. Dal Lago, U., Martini, S.: The weak lambda calculus as a reasonable machine. Theor. Comput. Sci. **398**(1-3), 32–50 (2008). https://doi.org/10.1016/J.TCS.2008.01.044
16. Danos, V., Herbelin, H., Regnier, L.: Game semantics & abstract machines. In: Proceedings, 11th Annual IEEE Symposium on Logic in Computer Science, New Brunswick, New Jersey, USA, July 27-30, 1996. pp. 394–405. IEEE Computer Society (1996). https://doi.org/10.1109/LICS.1996.561456
17. Friedman, D.P., Ghuloum, A., Siek, J.G., Winebarger, O.L.: Improving the lazy krivine machine. High. Order Symb. Comput. **20**(3), 271–293 (2007). https://doi.org/10.1007/S10990-007-9014-0
18. Grégoire, B., Leroy, X.: A compiled implementation of strong reduction. In: Wand, M., Jones, S.L.P. (eds.) Proceedings of the Seventh ACM SIGPLAN International Conference on Functional Programming (ICFP '02), Pittsburgh, Pennsylvania, USA, October 4-6, 2002. pp. 235–246. ACM (2002). https://doi.org/10.1145/581478.581501
19. Levy, P.B., Power, J., Thielecke, H.: Modelling environments in call-by-value programming languages. Inf. Comput. **185**(2), 182–210 (2003). https://doi.org/10.1016/S0890-5401(03)00088-9
20. Miller, D., Wu, J.H.: A positive perspective on term representation (invited talk). In: Klin, B., Pimentel, E. (eds.) 31st EACSL Annual Conference on Computer Science Logic, CSL 2023, February 13-16, 2023, Warsaw, Poland. LIPIcs, vol. 252, pp. 3:1–3:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2023). https://doi.org/10.4230/LIPICS.CSL.2023.3
21. Moggi, E.: Computational λ-Calculus and Monads. LFCS report ECS-LFCS-88-66, University of Edinburgh (1988), http://www.lfcs.inf.ed.ac.uk/reports/88/ECS-LFCS-88-66/ECS-LFCS-88-66.pdf
22. Moggi, E.: Computational lambda-calculus and monads. In: Proceedings of the Fourth Annual Symposium on Logic in Computer Science (LICS '89), Pacific Grove, California, USA, June 5-8, 1989. pp. 14–23. IEEE Computer Society (1989). https://doi.org/10.1109/LICS.1989.39155
23. Sands, D., Gustavsson, J., Moran, A.: Lambda Calculi and Linear Speedups. In: The Essence of Computation, Complexity, Analysis, Transformation. Essays Dedicated to Neil D. Jones. pp. 60–84 (2002). https://doi.org/10.1007/3-540-36377-7_4
24. Sestoft, P.: Deriving a lazy abstract machine. J. Funct. Program. **7**(3), 231–264 (1997). https://doi.org/10.1017/S0956796897002712
25. Wand, M.: On the correctness of the krivine machine. High. Order Symb. Comput. **20**(3), 231–235 (2007). https://doi.org/10.1007/S10990-007-9019-8
26. Wu, J.H.: Proofs as terms, terms as graphs. In: Hur, C. (ed.) Programming Languages and Systems - 21st Asian Symposium, APLAS 2023, Taipei, Taiwan, November 26-29, 2023, Proceedings. Lecture Notes in Computer Science, vol. 14405, pp. 91–111. Springer (2023). https://doi.org/10.1007/978-981-99-8311-7_5

Eval `(x.xx)((z.z)(z.z))`

	$\varepsilon \mid v_4[v_4 \leftarrow (\lambda v_1.v_5[v_5 \leftarrow v_1 v_1])v_6][v_6 \leftarrow (\lambda v_2.v_2)v_7][v_7 \leftarrow \lambda v_3.v_3] \mid \varepsilon$
-->sea$_1$	$\varepsilon \mid v_4[v_4 \leftarrow (\lambda v_1.v_5[v_5 \leftarrow v_1 v_1])v_6][v_6 \leftarrow (\lambda v_2.v_2)v_7] \mid [v_7 \leftarrow \lambda v_3.v_3]:\varepsilon$
-->m	$\varepsilon:(v_4[v_4 \leftarrow (\lambda v_1.v_5[v_5 \leftarrow v_1 v_1])v_6], v_6) \mid v_7 \mid [v_7 \leftarrow \lambda v_3.v_3]:\varepsilon$
-->sea$_3$	$\varepsilon \mid v_4[v_4 \leftarrow (\lambda v_1.v_5[v_5 \leftarrow v_1 v_1])v_7] \mid [v_7 \leftarrow \lambda v_3.v_3]:\varepsilon$
-->m	$\varepsilon:(v_4, v_4) \mid v_5[v_5 \leftarrow v_7 v_7] \mid [v_7 \leftarrow \lambda v_3.v_3]:\varepsilon$
-->e	$\varepsilon:(v_4, v_4) \mid v_5[v_5 \leftarrow (\lambda v_8.v_8)v_7] \mid [v_7 \leftarrow \lambda v_3.v_3]:\varepsilon$
-->m	$\varepsilon:(v_4, v_4):(v_5, v_5) \mid v_7 \mid [v_7 \leftarrow \lambda v_3.v_3]:\varepsilon$
-->sea$_3$	$\varepsilon:(v_4, v_4) \mid v_7 \mid [v_7 \leftarrow \lambda v_3.v_3]:\varepsilon$
-->sea$_3$	$\varepsilon \mid v_7 \mid [v_7 \leftarrow \lambda v_3.v_3]:\varepsilon$

Table 1. A screenshot from the Web interface. The active slice is highlighted.

A Implementation in OCaml

Compiling and Running the Prototype. The `README` explains how to compile and run the prototype. The code requires a working `dune` installation and it depends on the `js_of_ocaml` package, since it provides two different interfaces: a Read-Eval-Print-Loop (REPL) from the command line or a Web interface, i.e. an HTML+JavaScript page that performs all the computation on the client side. The two interfaces are functionally equivalent. The concrete syntax for λ-terms that is accepted by the implementation is also explained in the `README` file. A screenshot of the Web interface is shown in Table 1 where the λ-term $(\lambda x.xx)((\lambda z.z)(\lambda z.z))$ is transformed into a positive λ-term (via the transformation studied by Accattoli and Wu in [14]), namely (the variable names generated by the implementation are all numbered variants of v):

$$v_4[v_4 \leftarrow (\underbrace{\lambda v_1.v_5[v_5 \leftarrow v_1 v_1]}_{\lambda x.xx})v_6][v_6 \leftarrow (\underbrace{\lambda v_2.v_2}_{\lambda z.z})v_7][v_7 \leftarrow \underbrace{\lambda v_3.v_3}_{\lambda z.z}]$$

and then executed using the Sliced POM.

Code Structure. The implementation consists of five OCaml files. The first four, `utils.ml`, `lc.ml`, `repl.ml` and `main.ml`, are not interesting: they provide respectively utility functions, e.g. to output nicely both text and HTML, a parser and pretty-printer for λ-calculus terms, the command line REPL loop, and its equivalent for the Web interface. The final one, `spom.ml`, is the intersting one. It declares the data types for positive terms and Sliced POM states, pretty-printing functions for them, the transformation from λ-terms to positive terms (also called *crumbling*, following Accattoli et al. [5]), functions to immediately

substitute a variable for another in a term, α-renaming functions (i.e. copy), and the main loop of the Sliced POM.

Data Structures for Positive Terms. Positive terms are represented by the following data structure.

```
type term =
 | V of var
 | ESubst of term * var
and var =
 { name : int
 ; mutable subst : bite option
 ; mutable copying_to : var option
 }
and bite =
 | VV of var * var
 | AV of abst * var
 | A of abst
and abst = var * term
```

Variables are uniquely represented in memory by a single record var that is referenced by every variable occurrence (i.e. the reference v occurs in V v, VV(v,v'), VV(v',v), AV(a,v)). When a variable is bound by an explicit substitution, its field subst is set to Some b where b is the definiens. This allows to retrieve in $\mathcal{O}(1)$ the definiens of a variable given a variable occurrence. The term $t[x \leftarrow u]$ is represented by ESubst(t,x) since the record referenced by x contains u in its subst field.

We use integer numbers for variable names to make it simpler to generate fresh names. A variable such that name=n will be pretty-printed as v_n.

The copying_to field is set only during α-renaming (i.e. sharing preserving copy of terms): when a binder is encountered during copy for the first time, a new fresh variable is generated and the original bound variable is made to point to the new fresh variable using copying_to. When later an occurrence of the bound variable is found during the copy, the new variable reference is obtained from copying_to, preserving sharing.

Data Structures for Sliced POM States. Machine states are represented by the following data structures, that follow closely the definitions in the paper.

```
type env = var list
type slice = (term * var) list
type state = slice * term * env
```

Previous work on crumbled terms by Accattoli et al. [5,6] avoids the OCaml list type and implements environments (and other list-based data structures) by adding to variable nodes an optional field next pointing to the next entry in the list. This allows one to see the pairs term–env simply as a zipper data structure where variables in the term point to the next inner substitution and variables in the environment point to the next outer substitution. Sometimes an additional

prev pointer is also added, turning the lists into bidirectional lists to allow for constant time appending, which is required for some reduction machines.

In the implementation for this paper, we preferred to use standard OCaml lists to keep the code closer to the paper. Switching to the other representation would not change the computational complexity, but would diminish the constant for space usage.

Main Sliced POM Loop. We show here an excerpt of the code of the Sliced POM main loop, that implements the rules in the paper practically verbatim.

```
let rec eval ?last_rule_used (state:state) =
 ...
 match state with
   ...
 | (sl,ESubst(t,({subst=Some (A _);_} as subst)),env) ->
     (* sea1 *)
     eval ~last_rule_used:"sea1" (sl,t,subst::env)
 | (_,ESubst(_,({subst=Some(VV({subst=Some (A a);_},z));_} as var)),_)
       as state ->
     (* e *)
     var.subst <- Some(AV(alpha_abs a,z)) ;
     eval ~last_rule_used:(Utils.pad 3 "e") state
 | (sl,ESubst(t,({subst=Some(AV((y,ez),w));_} as x)),env) ->
     (* m *)
     x.subst <- None ;
     eval ~last_rule_used:(Utils.pad 3 "m")
      ((t,x)::sl,replace ~what:y ~with_:w ez,env)
 ...
```

You can see that the alpha_abs function is used in the exponential rule to rename an abstraction and that replace is used in the multiplicative rule to immediately substitue the variable y with the variable w. Those functions are simply defined by recursion on the syntax of positive terms as one would expect.

Note that all recursive calls are in tail position, making the recursive function equivalent to a simple loop. The optional last_rule_used parameter is used only for pretty-printing purposes, to show each machine transition to the user.

B Proofs Omitted from Sect. 3 (The Right Strategy)

Lemma 8 (Outer extension of inside-out is inside-out). *Let R_i be an inside-out right context. Then:*

1. *$R_i[x{\leftarrow}yz]$ is an inside-out right context;*
2. *If $x \notin \mathsf{afv}(R_i)$ then $R_i[x{\leftarrow}\lambda y.u]$ is an inside-out right context.*

Proof.

1. By induction on R_i. The base case is obvious.

- $R_i = R'_i\langle\langle\cdot\rangle[x'\leftarrow y'z']\rangle$ with $R'_i(y') \neq \lambda w.t$. By *i.h.*, $R'_i[x\leftarrow yz]$ is an inside-out right context. Then $R'_i\langle\langle\cdot\rangle[x'\leftarrow y'z']\rangle[x\leftarrow yz]$ is an inside-out right context.
- $R_i = R'_i\langle\langle\cdot\rangle[x'\leftarrow\lambda y'.u]\rangle$. By *i.h.*, $R'_i[x\leftarrow yz]$ is an inside-out right context. Then $R'_i\langle\langle\cdot\rangle[x'\leftarrow\lambda y'.u]\rangle[x\leftarrow yz]$ is an inside-out right context.

2. By induction on R_i. The base case is obvious.
   - $R_i = R'_i\langle\langle\cdot\rangle[x'\leftarrow y'z']\rangle$ with $R'_i(y) \neq \lambda w.t$. Since $x \notin \mathsf{afv}(R'_i)$, by *i.h.* we obtain that $R'_i[x\leftarrow\lambda y.u]$ is an inside-out right context. By hypothesis, $R'_i(y) \neq \lambda w.t$, and the hypothesis $x \notin \mathsf{afv}(R_i)$ implies that $y' \neq x$, so that $(R'_i[x\leftarrow\lambda y.u])(y) \neq \lambda w.t$. Then $R'_i\langle\langle\cdot\rangle[x'\leftarrow y'z']\rangle[x\leftarrow\lambda y.u]$ is an inside-out right context.
   - $R_i = R'_i\langle\langle\cdot\rangle[x'\leftarrow\lambda y'.u]\rangle$. Since $x \notin \mathsf{afv}(R'_i)$, by *i.h.* we obtain that $R'_i[x\leftarrow\lambda y.u]$ is an inside-out right context. Then $R'_i\langle\langle\cdot\rangle[x'\leftarrow\lambda y'.u]\rangle[x\leftarrow\lambda y.u]$ is an inside-out right context. □

Lemma 9 (Inner extension of outside-in is outside-in). *Let R_o be an outside-in right context. Then:*

1. *If $R_o(y) \neq \lambda w.t$ then $R_o\langle\langle\cdot\rangle[x\leftarrow yz]\rangle$ is an outside-in right context;*
2. *$R_o\langle\langle\cdot\rangle[x\leftarrow\lambda y.u]\rangle$ is an outside-in right context.*

Proof.

1. By induction on R_o. The base case is obvious.
   - $R_o = R'_o[x'\leftarrow y'z']$. By *i.h.*, $R'_o\langle\langle\cdot\rangle[x\leftarrow yz]\rangle$ is an outside-in right context. Then $R'_o\langle\langle\cdot\rangle[x\leftarrow yz]\rangle[x'\leftarrow y'z']$ is an outside-in right context.
   - $R_o = R'_o[x'\leftarrow\lambda y'.u]$ with $x' \notin \mathsf{afv}(R'_o)$. By *i.h.*, $R'_o\langle\langle\cdot\rangle[x\leftarrow yz]\rangle$ is an outside-in right context. By hypothesis, $R_o(y) \neq \lambda w.t$, which implies $x' \neq y$. Then $R'_o\langle\langle\cdot\rangle[x\leftarrow yz]\rangle[x'\leftarrow\lambda y'.u]$ is an outside-in right context.
2. By induction on R_o. The base case is obvious.
   - $R_o = R'_o[x'\leftarrow y'z']$. By *i.h.*, $R'_o\langle\langle\cdot\rangle[x\leftarrow\lambda y.u]\rangle$ is an ouside-in right context. Then $R'_o\langle\langle\cdot\rangle[x\leftarrow\lambda y.u]\rangle[x'\leftarrow y'z']$ is an outside-in right context.
   - $R_o = R'_o[x'\leftarrow\lambda y'.u]$ with $x' \notin \mathsf{afv}(R'_o)$. By *i.h.*, $R'_o\langle\langle\cdot\rangle[x\leftarrow\lambda y.u]\rangle$ is an ouside-in right context. Then $R'_o\langle\langle\cdot\rangle[x\leftarrow\lambda y.u]\rangle[x'\leftarrow\lambda y'.u]$ is an outside-in right context. □

Lemma 10. *Outside-in and inside-out right contexts coincide.*

Link to the original position of Lemma 2, p. 8

Proof.

- *Outside-in are inside-out.* Let R_o be an outside-in right context. By induction on R_o. The base case is obvious.
  - $R_o = R'_o[x\leftarrow yz]$. By *i.h.*, R'_o is an inside-out right context. By Lemma 8.1, $R'_o[x\leftarrow yz]$ is an inside-out right context.
  - $R_o = R'_o[x\leftarrow\lambda y.u]$ with $x \notin \mathsf{afv}(R_o)$. By *i.h.*, R'_o is an inside-out right context. By Lemma 8.2, $R'_o[x\leftarrow\lambda y.u]$ is an inside-out right context.
- *Inside-out are outside-in.* Let R_i be an inside-out right context. By induction on R_i. The base case is obvious.

- $R_i = R'_i\langle\langle\cdot\rangle[x{\leftarrow}yz]\rangle$ with $R'_i(y) \neq \lambda w.t$. By *i.h.*, R'_i is an outside-in right context. By Lemma 9.1, $R'_i\langle\langle\cdot\rangle[x{\leftarrow}yz]\rangle$ is an outside-in right context.
- $R_i = R'_i\langle\langle\cdot\rangle[x{\leftarrow}\lambda y.u]\rangle$. By *i.h.*, R'_i is an outside-in right context. By Lemma 9.2, $R'_i\langle\langle\cdot\rangle[x{\leftarrow}\lambda y.u]\rangle$ is an outside-in right context. □

Link to the original position of Lemma 3, p. 8

Lemma 11 (Basic properties of the right strategy).

1. Determinism*: if* $t \to_{\mathtt{r}} t_1$ *and* $t \to_{\mathtt{r}} t_2$ *then* $t_1 = t_2$*;*
2. No premature stops*: if* $t \to_{\mathtt{pos}} u$ *then* $t \to_{\mathtt{r}} r$ *for some* r*.*

Proof.

1. Let R and R' be the positions of the two redexes. Without loss of generality, we assume that R' strictly extends R inward, that is, there exists $O \neq \langle\cdot\rangle$ such that $R' = R\langle O\rangle$. Then we consider the different cases for the redex of position R and we shall derive every time a contradiction. The only possibility left shall be that $R = R'$. Cases of the redexes:
   - R is the position of a re-redex. Then $R' = O'\langle O''\langle O'''[x{\leftarrow}yz]\rangle[y{\leftarrow}\lambda w.t]\rangle$, which is not a right context because $x \in \mathsf{afv}(O''\langle O'''[x{\leftarrow}yz]\rangle)$. Absurd.
   - R is the position of a rm-redex. Then $R' = O'\langle O''[x{\leftarrow}(\lambda y.t)z]\rangle$, which is not a right context because no right contexts contains ESs of shape $[x{\leftarrow}(\lambda y.t)z]$.
2. Let O be the position of $t \to_{\mathtt{pos}} u$. If O is a right context the statement holds. Otherwise, let R be the maximum right context that is a prefix of O, that is, such that $O = R\langle O'\rangle$ for some O'. We show that it is the position of a redex. Let q be the sub-term isolated by R, that is, such that $t = R\langle q\rangle$. Cases of q:
   - $q = q'[x{\leftarrow}(\lambda y.q'')z]$. Then R is the position of a rm-redex.
   - $q = q'[x{\leftarrow}\lambda y.q'']$. Impossible, because then $R\langle[x{\leftarrow}\lambda y.q'']\rangle$ is a right context bigger than R and prefix of O, against maximality of R.
   - $q = q'[x{\leftarrow}yz]$. By maximality of R, one has that $R(y)$ is an abstraction, so that R is the position of a re-redex. □

C Proofs omitted from Sect. 5 (Preliminaries about Abstract Machines)

Lemma 12 (One-step simulation). *Let a machine* M *and a strategy* $\to_{\mathtt{str}}$ *be a distillery. For any state* $\mathtt{Q}$ *of* M*, if* $\overline{\mathtt{Q}} \to_{\mathtt{str}} u$ *then there is a state* $\mathtt{Q}'$ *of* M *such that* $\mathtt{Q} \rightsquigarrow^*_{\mathtt{sea}}\rightsquigarrow_{\mathtt{pr}} \mathtt{Q}'$ *and* $\overline{\mathtt{Q}'} = u$ *and such that the label of the principal transition and of the step are the same.*

Proof. For any state $\mathtt{Q}$ of M, let $\mathsf{nf}_{\mathtt{o}}(\mathtt{Q})$ be a normal form of $\mathtt{Q}$ with respect to $\rightsquigarrow_{\mathtt{sea}}$: such a state exists because search transitions terminate (Point 3 of Def. 5). Since $\rightsquigarrow_{\mathtt{sea}}$ is mapped on identities (by *search transparency*, *i.e.* Point 2 of Def. 5), one has $\overline{\mathsf{nf}_{\mathtt{o}}(\mathtt{Q})} = \overline{\mathtt{Q}}$. Since $\overline{\mathtt{Q}}$ is not $\to_{\mathtt{str}}$-normal by hypothesis, the halt property (Point 5) entails that $\mathsf{nf}_{\mathtt{o}}(\mathtt{Q})$ is not final, therefore $\mathtt{Q} \rightsquigarrow^*_{\mathtt{sea}} \mathsf{nf}_{\mathtt{o}}(\mathtt{Q}) \rightsquigarrow_{\mathtt{pr}} \mathtt{Q}'$ for some state $\mathtt{Q}'$, and thus $\overline{\mathtt{Q}} = \overline{\mathsf{nf}_{\mathtt{o}}(\mathtt{Q})} \to_{\mathtt{str}} \overline{\mathtt{Q}'}$ by principal projection (Point 1). By determinism of $\to_{\mathtt{str}}$ (Point 4), one obtains $\overline{\mathtt{Q}'} = u$. □

Theorem 5 (Sufficient condition for mechanical bisimulations). *Let a machine* M *and a strategy* $\to_{\mathtt{str}}$ *be a distillery. Then, they are mechanical bisimilar.*

Link to the original position of Thm. 2, p. 13

Proof. According to Def. 4, given a positive term t and an initial state $t \triangleleft \mathtt{Q}$, we have to show that:

1. *Runs to evaluations*: for any M-run $r : t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathsf{M}} \mathtt{Q}'$ there exists a $\to_{\mathtt{str}}$-evaluation $e : t \to^*_{\mathtt{str}} \overline{\mathtt{Q}'}$;
2. *Evaluations to runs*: for every $\to_{\mathtt{str}}$-evaluation $e : t \to^*_{\mathtt{str}} u$ there exists a M-run $r : t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathsf{M}} \mathtt{Q}'$ such that $\overline{\mathtt{Q}'} = u$;

Plus the principal matching constraint that shall be evident by the principal projection requirement and how the proof is built.

Proof of Point 1. By induction on $|r|_{\mathtt{p}} \in \mathbb{N}$. Cases:

- $|r|_{\mathtt{p}} = 0$. Then $r \colon t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathtt{sea}} \mathtt{Q}'$ and hence $\overline{\mathtt{Q}} = \overline{\mathtt{Q}'}$ by search transparency (Point 2 of Def. 5). Moreover, $t = \overline{\mathtt{Q}}$ since decoding is the inverse of initialization, therefore the statement holds with respect to the empty evaluation e with starting and end term t.
- $|r|_{\mathtt{p}} > 0$. Then, $r \colon t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathsf{M}} \mathtt{Q}'$ is the concatenation of a run $r' \colon t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathsf{M}} \mathtt{Q}''$ followed by a run $r'' \colon \mathtt{Q}'' \rightsquigarrow_{\mathtt{pr}} \mathtt{Q}''' \rightsquigarrow^*_{\mathtt{sea}} \mathtt{Q}'$. By *i.h.* applied to r', there exists an evaluation $e' \colon t \to^*_{\mathtt{str}} \overline{\mathtt{Q}'}$ satisfying the principal matching constraint. By principal projection (Point 1 of Def. 5) and search transparency (Point 2 of Def. 5) applied to r'', one obtains a one-step evaluation $e'' \colon \overline{\mathtt{Q}'} \to_{\mathtt{str}} \overline{\mathtt{Q}}$ having the same principal label of the transition. Concatenating e' and e'', we obtain an evaluation $e''' \colon t \to^*_{\mathtt{str}} \overline{\mathtt{Q}'} \to_{\mathtt{str}} \overline{\mathtt{Q}}$.

Proof of Point 2. By induction on $|e| \in \mathbb{N}$. Cases:

- $|e| = 0$. Then $t = u$. Since decoding is the inverse of initialization, one has $\overline{\mathtt{Q}} = t$. Then the statement holds with respect to the empty (*i.e.* without transitions) run r with initial (and final) state $\mathtt{Q}$.
- $|e| > 0$. Then, $e \colon t \to^*_{\mathtt{str}} u$ is the concatenation of an evaluation $e' \colon t \to^*_{\mathtt{str}} u'$ followed by the step $u' \to_{\mathtt{str}} u$. By *i.h.*, there exists a M-run $r' \colon t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathsf{M}} \mathtt{Q}''$ such that $\overline{\mathtt{Q}''} = u'$ and verifying the principal matching constraint. Since $\overline{\mathtt{Q}''} = u' \to_{\mathtt{str}} u$. By one-step simulation (Lemma 12), there is a state $\mathtt{Q}'$ of M such that $\mathtt{Q}'' \rightsquigarrow^*_{\mathtt{sea}} \rightsquigarrow_{\mathtt{pr}} \mathtt{Q}'$ and $\overline{\mathtt{Q}'} = u$, preserving the label of the step/transition. Therefore, the run $r \colon t \triangleleft \mathtt{Q} \rightsquigarrow^*_{\mathsf{M}} \mathtt{Q}'' \rightsquigarrow^*_{\mathtt{sea}} \rightsquigarrow_{\mathtt{pr}} \mathtt{Q}'$ satisfies the statement. □

D Proofs Omitted from Sect. 7 (Mechanical Bisimulations)

Theorem 6 (Distillery).

Link to the original position of Thm. 3, p. 15

1. *Principal projection:*

(a) *if* $\mathtt{Q} \rightsquigarrow_{\mathsf{e}} \mathtt{Q}'$ *then* $\overline{\mathtt{Q}} \to_{\mathsf{e}} \overline{\mathtt{Q}'}$.
(b) *if* $\mathtt{Q} \rightsquigarrow_{\mathsf{m}} \mathtt{Q}'$ *then* $\overline{\mathtt{Q}} \to_{\mathsf{m}} \overline{\mathtt{Q}'}$.
2. *Search transparency: if* $\mathtt{Q} \rightsquigarrow_{\mathtt{sea}_1,\mathtt{sea}_2,\mathtt{sea}_3} \mathtt{Q}'$ *then* $\overline{\mathtt{Q}} = \overline{\mathtt{Q}'}$.
3. *Search terminates:* $\rightsquigarrow_{\mathtt{sea}_1,\mathtt{sea}_2,\mathtt{sea}_3}$ *is terminating.*
4. *Halt: if* $\mathtt{Q}$ *is final then it is of the form* $(\epsilon, x, \mathtt{E})$ *and* $\overline{\mathtt{Q}} = \overline{\mathtt{E}}\langle x\rangle$ *is* $\to_{\mathsf{pos}}$*-normal.*

Proof.

1. (a) If $\mathtt{Q} = (\mathtt{S}, t[x{\leftarrow}yz], \mathtt{E}) \rightsquigarrow_{\mathsf{e}} (\mathtt{S}, t[x{\leftarrow}(\lambda w.u)z], \mathtt{E}) = \mathtt{Q}'$ with $\mathtt{E}(y) = \lambda w.u$ then $\mathtt{E} = \mathtt{E}' : [y{\leftarrow}\lambda w.u] : \mathtt{E}''$ for some $\mathtt{E}'$ and $\mathtt{E}''$. We proceed by induction on $\mathtt{S}$. Cases:
   - *Empty slice stack, i.e.* $\mathtt{S} = \epsilon$. We have:

$$\begin{array}{rl} & \overline{(\epsilon, t[x{\leftarrow}yz], \mathtt{E}' : [y{\leftarrow}\lambda w.u] : \mathtt{E}'')} \\ = & \overline{\mathtt{E}''}\langle\overline{\mathtt{E}'}\langle t[x{\leftarrow}yz]\rangle[y{\leftarrow}\lambda w.u]\rangle \\ \to_{\mathsf{e}} & \overline{\mathtt{E}''}\langle\overline{\mathtt{E}'}\langle t[x{\leftarrow}(\lambda w.u)z]\rangle[y{\leftarrow}\lambda w.u]\rangle \\ = & \overline{(\epsilon, t[x{\leftarrow}(\lambda w.u)^{\alpha}z], \mathtt{E}' : [y{\leftarrow}\lambda w.u] : \mathtt{E}'')} \end{array}$$

     Note that the exponential step applies because of:
     - The well-bound invariant (Lemma 4.2) that guarantees that $\mathtt{E}'$ does not capture y (because by the invariant there cannot be two ESs on the same name);
     - The contextual read-back invariant (Lemma 4.1) that guarantees that $\overline{\mathtt{E}'}$ and $\overline{\mathtt{E}''}$ are right contexts.
   - *Non-empty slice stack, i.e.* $\mathtt{S} = \mathtt{S}' : r[x'{\leftarrow}\cdot]$. Let $t = O\langle y'\rangle$. We have:

$$\begin{array}{rl} & \overline{(\mathtt{S}' : r[x'{\leftarrow}\cdot]\ , t[x{\leftarrow}yz]\ , \mathtt{E})} \\ = & \overline{(\mathtt{S}' : r[x'{\leftarrow}\cdot]\ , O\langle y'\rangle[x{\leftarrow}yz]\ , \mathtt{E})} \\ = & \overline{(\mathtt{S}'\ , O\langle r\{x'{\leftarrow}y'\}\rangle[x{\leftarrow}yz]\ , \mathtt{E})} \\ (\text{by } \textit{i.h.}) \to_{\mathsf{e}} & \overline{(\mathtt{S}'\ , O\langle r\{x'{\leftarrow}y'\}\rangle[x{\leftarrow}(\lambda w.u)^{\alpha}z]\ , [x{\leftarrow}yz] : \mathtt{E})} \\ = & \overline{(\mathtt{S}' : r[x'{\leftarrow}\cdot]\ , O\langle y'\rangle[x{\leftarrow}(\lambda w.u)^{\alpha}z]\ , [x{\leftarrow}yz] : \mathtt{E})} \\ = & \overline{(\mathtt{S}' : r[x'{\leftarrow}\cdot]\ , t[x{\leftarrow}(\lambda w.u)^{\alpha}z]\ , [x{\leftarrow}yz] : \mathtt{E})} \end{array}$$

   (b) Let $\mathtt{Q} = (\mathtt{S}, t[x{\leftarrow}(\lambda y.O\langle z\rangle)w], \mathtt{E}) \rightsquigarrow_{\mathsf{m}} (\mathtt{S} : t[x{\leftarrow}\cdot], O\langle z\rangle\{y{\leftarrow}w\}, \mathtt{E}) = \mathtt{Q}'$. We proceed by induction on $\mathtt{S}$. Cases:
   - *Empty slice stack, i.e.* $\mathtt{S} = \epsilon$. We have:

$$\begin{array}{rll} \overline{(\epsilon, t[x{\leftarrow}(\lambda y.O\langle z\rangle)w], \mathtt{E})} = & & \overline{\mathtt{E}}\langle t[x{\leftarrow}(\lambda y.O\langle z\rangle)w]\rangle \\ & \to_{\mathsf{m}} & \overline{\mathtt{E}}\langle O\langle t\{x{\leftarrow}z\}\rangle\{y{\leftarrow}w\}\rangle \\ & = & \overline{(\epsilon, O\langle t\{x{\leftarrow}z\}\rangle\{y{\leftarrow}w\}, \mathtt{E})} \\ & =_{y\notin\mathsf{fv}(t)} & \overline{(\epsilon : t[x{\leftarrow}\cdot], O\langle z\rangle\{y{\leftarrow}w\}, \mathtt{E})} \end{array}$$

     Note that $y \notin \mathsf{fv}(t)$ is ensured by the well-bound invariant (Lemma 4.2). Moreover, the contextual read-back invariant (Lemma 4.1) that guarantees that $\overline{\mathtt{E}}$ is a right context.

– *Non-empty slice stack*, *i.e.* $\mathtt{S} = \mathtt{S}' : u[x'\leftarrow\cdot]$. Let $t = O'\langle y'\rangle$. We have:

$$\begin{array}{rl}
 & \overline{(\mathtt{S}' : u[x'\leftarrow\cdot] \quad , t[x\leftarrow(\lambda y.O\langle z\rangle)w] \quad , \mathtt{E})} \\
= & \overline{(\mathtt{S}' : u[x'\leftarrow\cdot] \quad , O'\langle y'\rangle[x\leftarrow(\lambda y.O\langle z\rangle)w] \quad , \mathtt{E})} \\
= & \overline{(\mathtt{S}' \quad , O'\langle u\{x'\leftarrow y'\}\rangle[x\leftarrow(\lambda y.O\langle z\rangle)w] \quad , \mathtt{E})} \\
(\text{by } i.h.) \rightarrow_{\mathsf{m}} & \overline{(\mathtt{S}' : O'\langle u\{x'\leftarrow y'\}\rangle[x\leftarrow\cdot] \, , O\langle z\rangle\{y\leftarrow w\} \quad , \mathtt{E})} \\
=_{y\notin \mathsf{fv}(O'\langle u\{x'\leftarrow y'\}\rangle)} & \overline{(\mathtt{S}' \quad , O\langle O'\langle u\{x'\leftarrow y'\}\rangle\{x\leftarrow z\}\rangle\{y\leftarrow w\} \, , \mathtt{E})} \\
=_{x\notin \mathsf{fv}(u)} & \overline{(\mathtt{S}' : u[x'\leftarrow\cdot] \quad , O\langle O'\langle y'\rangle\{x\leftarrow z\}\rangle\{y\leftarrow w\} \quad , \mathtt{E})} \\
= & \overline{(\mathtt{S}' : u[x'\leftarrow\cdot] \quad , O\langle t\{x\leftarrow z\}\rangle\{y\leftarrow w\} \quad , \mathtt{E})} \\
=_{y\notin \mathsf{fv}(t)} & \overline{(\mathtt{S}' : u[x'\leftarrow\cdot] : t[x\leftarrow\cdot] \quad , O\langle z\rangle\{y\leftarrow w\} \quad , \mathtt{E})}
\end{array}$$

Note that the three conditions about variables justifying the equalities are ensured by the well-bound invariant (Lemma 4.2).

2. Cases of search transitions:
   – $\mathtt{Q} = (\mathtt{S}, t[x\leftarrow\lambda y.u], \mathtt{E}) \rightsquigarrow_{\mathtt{sea}_1} (\mathtt{S}, t, [x\leftarrow\lambda y.u] : \mathtt{E}) = \mathtt{Q}'$. Cases of $\mathtt{S}$:
   • *Empty slice stack*, *i.e.* $\mathtt{S} = \epsilon$. Then:

$$\begin{array}{rl}
\overline{(\epsilon, t[x\leftarrow\lambda y.u], \mathtt{E})} = \overline{\mathtt{E}}\langle t[x\leftarrow\lambda y.u]\rangle & \\
= \overline{[x\leftarrow\lambda y.u] : \mathtt{E}}\langle t\rangle = \overline{(\epsilon, t, [x\leftarrow\lambda y.u] : \mathtt{E})}. &
\end{array}$$

   • *Non-empty slice stack*, *i.e.* $\mathtt{S} = \mathtt{S}' : r[z\leftarrow\cdot]$. Let $t = O\langle w\rangle$. We have

$$\begin{array}{rl}
 & \overline{(\mathtt{S}' : r[z\leftarrow\cdot] \, , t[x\leftarrow\lambda y.u] \quad , \mathtt{E})} \\
= & \overline{(\mathtt{S}' : r[z\leftarrow\cdot] \, , O\langle w\rangle[x\leftarrow\lambda y.u] \quad , \mathtt{E})} \\
= & \overline{(\mathtt{S}' \quad , O\langle r\{z\leftarrow w\}\rangle[x\leftarrow\lambda y.u] \quad , \mathtt{E})} \\
=_{i.h.} & \overline{(\mathtt{S}' \quad , O\langle r\{z\leftarrow w\}\rangle \quad , [x\leftarrow\lambda y.u] : \mathtt{E})} \\
= & \overline{(\mathtt{S}' : r[z\leftarrow\cdot] \, , O\langle w\rangle \quad , [x\leftarrow\lambda y.u] : \mathtt{E})} \\
= & \overline{(\mathtt{S}' : r[z\leftarrow\cdot] \, , t \quad , [x\leftarrow\lambda y.u] : \mathtt{E})}
\end{array}$$

   – $\mathtt{Q} = (\mathtt{S}, t[x\leftarrow yz], \mathtt{E}) \rightsquigarrow_{\mathtt{sea}_2} (\mathtt{S}, t, [x\leftarrow yz] : \mathtt{E}) = \mathtt{Q}'$ with either $y \notin \mathsf{dom}(\mathtt{E})$ or $\mathtt{E}(y) \neq \lambda w.u$. Cases of $\mathtt{S}$:
   • *Empty slice stack*, *i.e.* $\mathtt{S} = \epsilon$. Then:

$$\overline{(\epsilon, t[x\leftarrow yz], \mathtt{E})} = \overline{\mathtt{E}}\langle t[x\leftarrow yz]\rangle = \overline{[x\leftarrow yz] : \mathtt{E}}\langle t\rangle = \overline{(\epsilon, t, [x\leftarrow yz] : \mathtt{E})}.$$

   • *Non-empty slice stack*, *i.e.* $\mathtt{S} = \mathtt{S}' : u[w\leftarrow\cdot]$. Let $t = O\langle x'\rangle$. We have:

$$\begin{array}{rl}
 & \overline{(\mathtt{S}' : u[w\leftarrow\cdot] \, , t[x\leftarrow yz] \quad , \mathtt{E})} \\
= & \overline{(\mathtt{S}' : u[w\leftarrow\cdot] \, , O\langle x'\rangle[x\leftarrow yz] \quad , \mathtt{E})} \\
= & \overline{(\mathtt{S}' \quad , O\langle u\{w\leftarrow x'\}\rangle[x\leftarrow yz] \quad , \mathtt{E})} \\
=_{i.h.} & \overline{(\mathtt{S}' \quad , O\langle u\{w\leftarrow x'\}\rangle \quad , [x\leftarrow yz] : \mathtt{E})} \\
= & \overline{(\mathtt{S}' : u[w\leftarrow\cdot] \, , O\langle x'\rangle \quad , [x\leftarrow yz] : \mathtt{E})} \\
= & \overline{(\mathtt{S}' : u[w\leftarrow\cdot] \, , t \quad , [x\leftarrow yz] : \mathtt{E})}
\end{array}$$

898 – $\mathtt{Q} = (\mathtt{S} : t[x\leftarrow\cdot], z, \mathtt{E}) \rightsquigarrow_{\mathtt{sea}_3} (\mathtt{S}, t\{x\leftarrow z\}, \mathtt{E}) = \mathtt{Q}'$. By the very definition of
899 read-back $(\mathtt{S} : t[x\leftarrow\cdot], z, \mathtt{E}) \coloneqq (\mathtt{S}, t\{x\leftarrow z\}, \mathtt{E})$, the statement holds.
900 3. Any $\rightsquigarrow_{\mathtt{sea}_1,\mathtt{sea}_2,\mathtt{sea}_3}$-sequence is bound by the sum of the size of the active
901 code plus the sizes of the terms in the slice stack because:
902 – $\rightsquigarrow_{\mathtt{sea}_1}$ and $\rightsquigarrow_{\mathtt{sea}_2}$ consume ESs from the active code,
903 – $\rightsquigarrow_{\mathtt{sea}_3}$ consumes an entry from the slice when there are no ESs left on
904 the active code.
905 4. Let $\mathtt{Q} = (\mathtt{S}, t, \mathtt{E})$ be a final state. First, the active code t has be a variable
906 x. Otherwise, one among transitions $\mathtt{sea}_1, \mathtt{sea}_2, \mathtt{e}, \mathtt{m}$ would apply. Therefore,
907 the slice stack $\mathtt{S}$ has to be empty, otherwise transition $\mathtt{sea}_3$ would apply.
By the contextual read-back invariant (Lemma 4.1), $\overline{\mathtt{E}}$ is a right context,
which implies that $\overline{\mathtt{Q}} = \overline{\mathtt{E}}\langle x\rangle$ is $\rightarrow_{\mathtt{pos}}$-normal. □